

FEATURED PROJECT 03

IoT Home Security System

ESP32-CAM Sensor Integration and Telegram-Based Remote Monitoring

CONTEXT
Bachelor's Project

ROLE
Team Co-Developer

APPLICATION
Home Security

PERIOD
2021–2022

Project Overview

Co-developed and physically integrated a model-home security prototype combining an **AI Thinker ESP32-CAM**, camera-based evidence capture, intrusion sensors, environmental hazard detection, Wi-Fi connectivity, and Telegram-based remote monitoring. The system issued event-specific alerts for door, motion, flame, and gas/smoke conditions. Motion and door events also triggered image capture, while authenticated bot commands enabled individual monitoring functions, camera flash control, and on-demand photography.

Verified engineering work

- Assembled the ESP32-CAM, AM312 PIR sensor, magnetic door contact, flame detector, gas/smoke sensor, level shifting, and breadboard power hardware into a physical model-house demonstrator.
- Configured the OV2640 camera, Wi-Fi connection, and Telegram Bot API workflow using Arduino-compatible firmware and supporting libraries.
- Implemented event-response paths for remote alerts, on-demand image capture, camera-flash control, and independent arming or disarming of motion, door, flame, and gas/smoke monitoring.
- Adapted the planned gas-sensing hardware from MQ6 to **MQ5** when the original component was unavailable.
- Performed qualitative, scenario-based tests on the physical prototype; no Proteus simulation was used for the ESP32-CAM and Telegram workflow.

System at a glance

Intrusion monitoring

PIR motion and magnetic door sensing.

Visual evidence

OV2640 capture for motion and door events.

Environmental safety

MQ5 gas/smoke and optical flame detection.

Remote interaction

Telegram alerts, arming controls, and photo requests.



Completed model-home prototype with the ESP32-CAM positioned for visual monitoring.

CORE ENGINEERING SCOPE

Integrating sensor inputs, camera capture, Wi-Fi messaging, and physical prototype packaging into a single event-driven demonstrator.

TECHNOLOGIES

ESP32-CAM

OV2640

Embedded C++

Arduino IDE

Wi-Fi

Telegram Bot API

AM312 PIR

Magnetic Door Sensor

MQ5 Gas Sensor

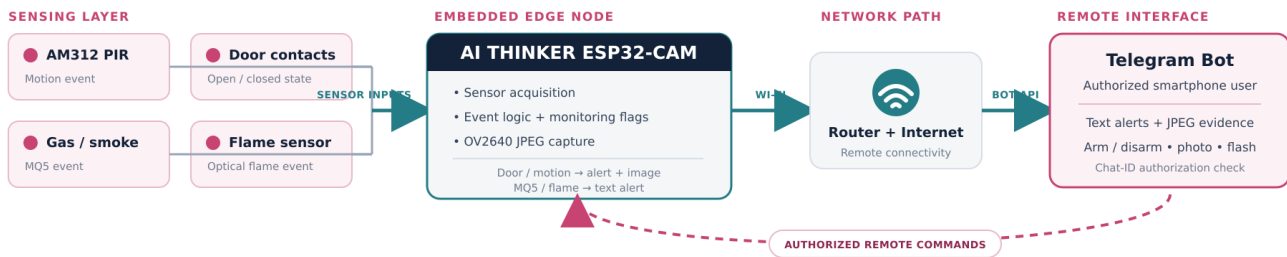
Flame Sensor

Sensor Integration

ARCHITECTURE AND VALIDATION

From Sensor Event to Local Decision and Remote Evidence

IMPLEMENTED SYSTEM ARCHITECTURE



Implemented event flow from local sensing and ESP32-CAM decision logic to Wi-Fi messaging and Telegram interaction.

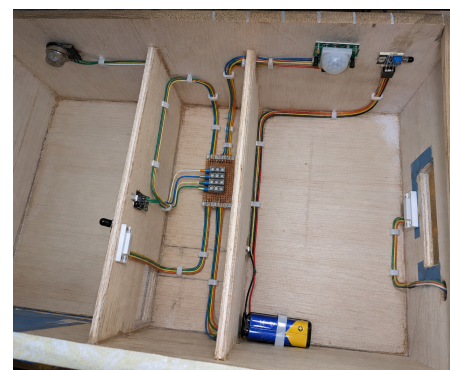
Embedded hardware and firmware

The ESP32-CAM served as the central sensing, control, imaging, and communication node. Digital inputs monitored motion, door state, and flame status, while the gas/smoke channel was connected through the prototype's level-shifting and power interface. The OV2640 camera produced JPEG frames for event-triggered or user-requested transmission.

Arduino-compatible C++ firmware used `esp_camera`, `WiFiClientSecure`, `UniversalTelegramBot`, and `ArduinoJson`. The firmware polled the bot for commands, checked the requester's chat ID, maintained independent monitoring flags, read the sensors, and sent either a text alert or a captured image according to the event type.

Communication and control flow

1. Acquire a sensor state or receive an authorized Telegram command.
2. Check whether the corresponding monitoring function is armed.
3. Classify the event as intrusion-related or environmental.
4. Capture a JPEG image for door or motion events when required.
5. Send the event message and available image through the bot interface.



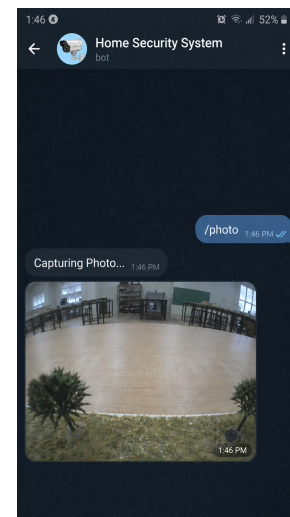
Interior installation showing routed wiring, door contacts, PIR, flame and MQ5 modules, and the centralized perfboard interconnect.

IMPLEMENTATION BASIS

The wiring and firmware baseline were adapted from the published make2explore ESP32-CAM / Telegram reference design. The verified project work is therefore presented as team-based physical integration, configuration, component adaptation, and functional testing.

RESULTS AND VALIDATION

The report documents a physical model-home prototype and states that MQ5 and flame events produced Telegram alerts, while PIR and door events produced alerts with images. Validation was qualitative: no latency, accuracy, false-alarm rate, calibration, endurance, or power measurements were reported.



Authorized Telegram photo request and returned camera image.

PROPOSED 2026 ARCHITECTURE EVOLUTION — NOT YET IMPLEMENTED

- Add deterministic sensor fusion, operating modes, and alert-severity logic so isolated triggers do not all produce the same response.
 - Make safety local-first with a siren, status indication, backup power, event logging, watchdog recovery, and sensor diagnostics.
 - Move event-triggered vision to an ESP32-S3 for lightweight local person detection; evaluate STM32N6 only if larger vision models are justified.
 - Protect the device lifecycle with encrypted credentials, authenticated communication, secure boot, and signed OTA updates with rollback.
- Critical gas and fire decisions should remain deterministic on the microcontroller. Edge AI belongs only in the visual-verification path, where it can reduce irrelevant image transmission without becoming a safety dependency.*